CHAPTER - 1
INTRODUCTION

1.1 OVERVIEW

Virtualization has a revolutionary place in contemporary computing where a single physical computer can be configured to host several virtual machines (VMs) and each will be an independent computing environment. Through this mechanism, hardware resources can be better utilized, system administration can be simplified, and software development and testing can be conducted in isolation without hardware replication.

The hypervisor alternatively referred to as virtual machine monitor (VMM) is the central software component that makes virtualization possible. It is an abstraction layer between hardware and operating systems, enabling more than one instance of an OS to co-exist and run on one host system at the same time. Hypervisors are classified into two general categories:

- Type-1 (bare-metal): Run natively on hardware and are installed in enterprise-class environments to offer maximum performance and isolation.
- Type-2 (hosted): Operate on a host operating system and are normally deployed in development and desktop setups due to the ease of use.

This project aims to develop a Type-2 hypervisor on the basis of Linux's native virtualization support - KVM (Kernel-based Virtual Machine)- and QEMU (Quick Emulator) for hardware emulation. Instead of using commercial or open-source pre-packaged programs like VMware, VirtualBox, or Hyper-V, a hypervisor has been built that talks directly to the native virtualization stack, giving complete control over resource management, virtual CPU allocation, memory allocation, and device emulation.

The project goes beyond virtualization utilization alone - it's about comprehending its underlying structure. It focuses on immediate virtualization logic and management system development, providing learning value and insight into hypervisor operation.

The functionalities provided are virtual machine lifecycle management (creation, start, shutdown, and deletion), image-based provisioning, boot from ISO support, and system monitoring capability—achieved by low-level libvirt and QEMU command implementation.

1.2 AIM OF THE PROJECT

The main objective of this project is to outline the architecture and behavior of hypervisors through the construction of a minimal but working Type-2 hypervisor that provides end-to-end virtual machine management on a Linux host. The project takes a hands-on, code-oriented approach that allows direct interaction with concepts such as virtualization modes, CPU emulation, memory mapping, and I/O management.

The project is a response to a realization that modern virtualization software presents itself as a black box system to students. Although these systems are excellent in terms of performance and functionality, they limit learning possibilities with low-level behavior concealment. The project fills this learning gap by providing direct access to hypervisor APIs and exposing VM control flow mechanics.

General Objectives:

- To understand hypervisor operation via direct utilization as opposed to standalone usage.
- To investigate KVM's contribution towards enabling kernel-level virtualization and the complementary role of QEMU in device emulation and operating system instantiation.
- To design an integrated management interface to allow user interaction with VMs through simplified, direct inputs for enhancing usability and test flexibility.
- To apply virtual CPU and memory allocation logic within the context of hardware-level virtualization principles.
- To stress isolation and security between guest and host systems.

Specific Objectives:

- Create a lightweight, pedagogical Type-2 hypervisor for Linux host platforms.
- Allow dynamic creation, management, and shutdown of virtual machines.
- Leverage Intel VT-x or AMD-V hardware acceleration through KVM to provide performance-efficient virtualization.
- Enforce all virtual machine lifecycle operations including initiation, graceful shutdown, forced shutdown, and deletion.
- Utilize libvirt and system tools for disk image management and VM creation from ISO.
- Provide VM isolation with well-delineated memory and resource quotas from the host machine.
- Offer a natural operation interface to facilitate testing and incremental learning.

The system has different components:

1. libvirt API: The design employs libvirt as the abstraction layer for virtualization management, providing a single API for KVM/QEMU interaction. This provides for uniform operations across virtual domains regardless of low-level virtualization technology details.
2. QEMU Device Emulation: While KVM manages virtualization of CPU and memory management, QEMU is utilized to complement it by providing device emulation of virtual hardware devices like disk controllers, network interfaces, and graphic adapters.
3. TigerVNC Integration: Graphical consoles for virtual machines connect to the hypervisor using the VNC protocol, and an automated VNC port is available for every VM for remote management and control.
4. VIRTIO Paravirtualized Drivers: For Windows installations, the system employs VIRTIO drivers, which greatly enhance I/O performance by enabling guest operating systems to interact with the hypervisor instead of trapping all I/O operations to the host.

CHAPTER - 2
LITERATURE REVIEW

This review of literature identifies existing research in the areas of hypervisor technology, security, and virtualization implementations. The research discussed herein serves as a basis for understanding hypervisor design, security issues, and suggested remedies in modern virtualization environments.

2.1 HYPERVISOR SECURITY AND ARCHITECTURE

Szefer and Lee (2012) introduced H-SVM, a hardware-based approach to enhance hypervisor security . H-SVM utilizes hardware isolation mechanism for protecting virtual machines against hypervisor-targeted attacks unlike the software-based methods. Their experimental results proved that this approach achieves good security guarantees with the least overheads possible. H-SVM is a major improvement in the field of virtualization security because it solves the [1] imperative flaw of hypervisor compromise without compromising the efficiency of the system.

Several research contributions have extended the hardware-assisted security concept, among which are a new hypervisor-driven approach to ensuring data confidentiality at the level of a page proposed by Yang and Shin (2008) [2]. Their approach secures sensitive application data without necessitating the applications' or the operating system's changes. Although application of the techniques did prove to be functionality safe with negligible impact on performance, authors realized that additional efficiency enhancements were necessary to make the solution more viable for generalized use.

2.2 VULNERABILITY ASSESSMENT AND THREAT MODELING

Aalam et al (2021) did a comprehensive review of challenges of hypervisor and virtual machine security . Their work finds chief threats to virtualized settings and assesses the existing protection mechanisms. The authors underlined that even though the current security solutions can provide better protective measures, the dynamic character of the threat landscape [3] requires keeping an eye on more effective defense strategies. This review illustrates the discrepancy between the current security implementations and the growing threats in the virtualization technology.
Citing an assessment of hypervisor vulnerabilities (Thongthua, 2016), cataloging major security risks and weaknesses from the existing protection systems, Thongthua and Ngamsuriyaroj (2016)

conducted an assessment of hypervisor vulnerabilities. The results of their findings indicated deep gaps in previous defense structures [4] and highlighted the immediate need for improved security measures against the forceful attacks. The work serves as a source of valuable information on the landscape of vulnerability, which must be taken care of in the contemporary hypervisor designs.

2.3 CODE PROTECTION AND EXECUTION CONTROL

Seshadri et al. (2007) devised SecVisor, a light weighted hypervisor which focuses on the security to make sure only the authorized code executes in kernel mode . Such an approach is successful in avoiding the execution of malicious code without compromising the acceptable performance of the system. Their experimental results validated SecVisor's capability to increase the kernel's security in minimal overhead, and it was, nonetheless, highlighted that further development will be needed to tackle the forthcoming security issues and enhance compatibility with the current systems[5].

Based on this concept, a hypervisor-based technique for code protection against other's modification as well as attacks was introduced by Kiperberg, et al. (2019). Their approach is always watching code execution to [6] dictate security policies with little or no performance overhead. While their solution worked well in a controlled environment, the authors found possibilities to enhance the efficiency and the ability to respond to developing threats.

2.4 FAULT TOLERANCE AND RELIABILITY

Bressoud and Schneider (1996) pioneered on hypervisor-based mechanisms for fault tolerance striving to sustain operations of the system irrespective of failures of hardware. The process of fault detection and recovery is quite refined in their approach and ensures reliability [7]. Although their work proved the essential feasibility of fault tolerance in hypervisor, they revealed the efficiency concerns and failure scenario handling limitations that remain to be solved in modern implementations.

2.5 GAPS IN CURRENT RESEARCH

1. Integration of Security and Performance: Although each study can treat either security or performance, not many can integrate robust security capabilities with high-performance virtualization.

2. Educational Hypervisor Implementations: Comparatively less effort has gone into creating educational hypervisors that expose the internal details without affecting performance.
3. Simplified Management Interfaces: Most of the research focuses on technical deployments rather than creating easy-to-use management interfaces for administering the hypervisor.
4. Open-Source Alternatives to Commercial Solutions: More open-source hypervisor implementations are needed to provide the openness and control of commercial solutions without the ensuing complexity.
5. Practical Implementation Guides: The literature lacks comprehensive implementation guides for hypervisors that bridge the gap between theoretical concepts and practical development phases.

Table 2.1.0 Summary of Literature

Sr No.	**Author(s)**	**Research Focus**	**Technology Used**	**Outcomes**	**Limitations**
[1]	J. Szefer, R. B. Lee	Hypervisor security via hardware isolation	H-SVM	Strong protection with minimal performance impact	Limited to hardware-supported virtualization
[2]	J. Yang, K. G. Shin	Data secrecy using hypervisor memory control	Hypervisor-based access control	Ensures security without modifying OS/apps	Needs optimization for efficiency
[3]	Z. Aalam, V. Kumar, S. Gour	Review of hypervisor and VM security	Various security mechanisms	Identifies threats, calls for stronger defenses	Further research needed for evolving threats
[4]	A. Thongthua, S. Ngamsuriyaroj	Assessment of hypervisor vulnerabilities	N/A (Review study)	Highlights need for stronger security measures	Calls for more effective, adaptive solutions

[5]	A. Seshadri et al.	Kernel code integrity protection	SecVisor	Prevents malicious code with minimal overhead	Needs broader compatibility with modern systems
[6]	M. Kiperberg et al.	Protecting code execution via hypervisor	Hypervisor-based monitoring	Prevents unauthorized code changes	Requires efficiency improvements
[7]	T. C. Bressoud, F. B. Schneider	Hypervisor-based fault tolerance	Hypervisor-based recovery	Ensures reliability despite failures	Needs improvements for complex failures

The current project addresses these gaps by developing a transparent, educational Type-2 hypervisor that exposes low-level virtualization mechanisms while providing a functional management interface. By focusing on both the implementation details and usability aspects, this work contributes to bridging the divide between theoretical virtualization concepts and practical hypervisor development.

CHAPTER - 3
PROBLEM STATEMENT AND OBJECTIVE

3.1 PROBLEM STATEMENT

Virtualization software is an integral component of computing today, utilized for server consolidation, cloud computing, and secure testing of software. Although extremely popular and well-established to employ robust hypervisors such as VMware, VirtualBox, and Microsoft Hyper-V, these are black-box systems hiding the low-level mechanisms from the users. This limits learning and teaching experiences of students, researchers, and hobbyists interested in learning about or experimenting with low-level virtualization processes such as memory mapping, CPU virtualization, interrupt management, and VM isolation.

Moreover, the majority of hypervisors available today are heavyweight, closed-source, or too complex to learn or to modify. This lack of low-level, low-barrier, hands-on tools is a huge gap in virtualization learning and experimentation. The lack of a minimal but functional hypervisor that talks directly to virtualization extensions (e.g., Intel VT-x or AMD-V) and exposes critical pieces—such as virtual CPU initialization and memory translation—makes it difficult for students to learn the basic concepts of virtual machine execution.

This project fills this gap by running a lightweight Type-2 hypervisor on top of KVM (Kernel-based Virtual Machine) and QEMU on a Linux host. It will give users an operational, editable starting point for virtual machine management, allowing users to work directly with the low-level virtualization API and be aware of what is happening within the internal workings of the hypervisor. Functionality is not the sole purpose, but also educational transparency and modularity.

3.2 OBJECTIVES

The project to be proposed intends to address the issues of education and access by creating a mini Type-2 hypervisor using Kernel-based Virtual Machine (KVM), Quick Emulator (QEMU), and TigerVNC for VM visualization. The objectives are as follows:

- To develop and run a light-weight Type-2 hypervisor on a Linux host, using KVM for hardware virtualization and QEMU for device emulation in an attempt to gain maximum performance.

- To enable the setup, running, and control of virtual machines along with multi-guest OS support while allowing tunable resources like memory, virtual CPUs, and disk space.
- To offer full VM lifecycle operations, including start, graceful stop, force off, and delete, with effective error handling and resource management.
- To offer safe isolation between guest VMs and host system to safeguard system integrity and prevent unauthorized access or resource conflicts.
- To include TigerVNC for remote graphical display of VM user interfaces, so users may interact with guest operating systems without interruption.
- To facilitate ISO-based VM provisioning and disk image management using libvirt, to facilitate flexible and user-friendly VM installation for educational purposes.

This project is an attempt to solve the problems exposed here by designing an educational Type-2 hypervisor that simplifies the understanding of the virtualization technology due to its open implication and minimalistic design. By developing a hypervisor that works directly with Linux's KVM and QEMU infrastructure, the project will offer unfettered visibility into virtualization action, without having to trade functional ability for the virtual machine management. Through accomplishing these goals, the project will eliminate the distance between the abstract virtualization theory and the reality of its usage, becoming an invaluable source of knowledge on various subjects of education, research, and hypervisor technologies' advancements. The outcome system will not only be a working hypervisor but also an open system through which it will be possible to experiment and find out how virtualization technology works.

CHAPTER - 4
METHODOLOGY OF THE PROJECT

4.1 ARCHITECTURE AND SYSTEM MODEL

The proposed hypervisor implementation co-opts a well-layered, hyper-structured architecture. being created with the purpose of consolidating a collection of virtualization elements into a consummate and operationally potent virtual machine (VM) management system. This architecture is hierarchical in design as each layer and the component that received a unique task to guarantee smooth running, scalability, and accessibility for educational purposes. The system uses the advantages of a Type-2 hypervisor model. working on top of a Linux host operating system; this means that it is able to take advantage of the host when it comes to use of resources. management capabilities, device drivers and kernel-level services, but isolated environment for virtual machines. The architecture is designed in a way that prioritizes transparency so that it is able to providende users to communicate with the low-level mechanisms of virtualization, and understand them. including cutting-edge functionality like remote graphical access and I/O optimization for a variety of guest operating systems, including Windows.

4.1.1 Core Virtualization Infrastructure

The hypervisor base includes the virtualization infrastructure i.e., the hypervisor-dependent machine, hypervisor-independent machine and the Bi-Window machine. two primary constituents: Kernel-based Virtual Machine (KVM) and Quick EMUlator (QEMU). The two constituents are used at the same time in terms of the provision of the virtualization and the hardware. emulation features used in hosting isolated guest operating systems on the host.

4.1.1.1 KVM (Kernel-based Virtual Machine):

- It is a component of kernel virtualization that is embedded deep into the Linux kernel that actually makes the regular Linux kernel into a type-2 hypervisor running virtual machines with almost native performance. KVM leverages the availability of technologies, which help in the hardware-assisted virtualization capabilities to perform effective execution of the guest operating systems. KVM utilizes the extensions of the processor such as Intel VT-x (Virtual machine extensions) or AMD-V (secure virtual machine) in order to run the guest code directly on the physical CPUs in the isolated execution environment. These

extensions provide the basic virtualization primitives, like Virtual Machine Control Structure (VMCS) for Intel CPUs, or the Virtual Machine Control Block (VMCB) for AMD CPUs. Such structures allow KVM to manage virtual CPU states, manage guest-to-host or host-to-guest mode switching and manage isolation between virtual machines and host system.

- KVM also goes a long way in offering highly supportive functions in regards to I/O virtualization, an important consideration in ensuing that virtual machines have zero - defined access to devices. It supports the features like Intel VT-d (Virtualization Technology for Directed I/O) and AMD-Vi (I/O Virtualization), which help provide direct device sharing with the virtual machines. Direct assignment, or passthrough for short, lets a VM directly use physical hardware devices like network or storage controllers, thereby not using any software emulation interfaces, and improving I/O performance by eliminating latency and overhead. KVM also supports VirtIO, a paravirtualized I/O system which standardized support is perfect for supporting guest operating systems such as windows. In response, KVM has adopted the VirtIO design to provide a set of virtual device drivers (e.g., VirtIO-net for network, VirtIO-block for storage) that enable guest OSes to communicate with the host efficiently, but realising near-native I/O performance while still maintaining isolation. For the Windows guests VirtIO drivers are required to provide the optimized performance in terms of the network, storage, and graphical environment in order to make it possible for the VM to perform the I/O operations without expending significant overhead in comparison with the fully emulated devices.
- Virtualization of memory is another important job of KVM and it happens with the assistance of Extended Page Tables (EPT) or Nested Page Tables (NPT) for Intel or AMD processors correspondingly. These hardware features enable effective mapping of guest virtual addresses to host physical addresses, minimizing software-based shadow page tables and enhancing memory access performance. CPU scheduling for virtual machines is also handled by KVM, providing fair and efficient allocation of CPU time to virtual CPUs (vCPUs) on the host's physical processors while keeping VMs isolated from each other to avoid resource contention

4.1.1.2 QEMU (Quick Emulator):

QEMU functions as the hardware emulation layer, complementing KVM by providing a comprehensive set of virtual hardware devices for guest operating systems. While KVM handles CPU and memory virtualization, QEMU emulates the physical hardware environment required by the guest OS, including storage devices, network interfaces, graphical devices, and input/output peripherals. QEMU operates in user space and interfaces with KVM through the

/dev/kvm device file, allowing it to offload CPU-intensive tasks to KVM while focusing on device emulation.

QEMU is tasked with generating and manipulating virtual disk images, usually in qcow2 format, that include snapshots, thin provisioning, and compression to be suitable for testing and learning. For instance, when the user is opening a new VM, QEMU will create a qcow2 disk image that will be the guest OS virtual hard drive, with sizes customizable through user inputs (e.g., 20 GB). QEMU also provides booting off of ISO files, emulating a virtual CD-ROM drive in order to install guest operating systems. On the I/O front, QEMU provides VirtIO devices, which are utilized in order to give paravirtualized network and storage interfaces to the guest operating system. On Windows guests, QEMU sets up VirtIO-block devices for storage and VirtIO-net for networking, calling for the VirtIO drivers to be installed to provide the best performance.
Moreover, QEMU handles graphical display for virtual machines, including support for protocols like VNC and SPICE. Within this design, QEMU is set to provide VNC support, which works with TigerVNC to enable remote graphical access to the VM. QEMU allocates a special VNC port to each VM (e.g., autoport with listen address 0.0.0.0), and users can access the VM's graphical console remotely via the network. QEMU also simulates a virtual network interface based on the VirtIO model, linking the VM to the host system's default network (e.g., through a NAT-based virtual network), so that the guest operating system can obtain network connectivity for purposes like software updates or remote login.

4.1.2 API and Management Layer

The management layer and API acts as the middleman between the core virtualization infrastructure and the hypervisor implementation, abstracting low-level operations and presenting a standard interface for VM management.

4.1.2.1 Libvirt:

Libvirt acts as a strong, standardized API and management tool that enables the hypervisor implementation to communicate effectively with the underlying virtualization infrastructure, namely KVM and QEMU. Libvirt offers a single interface to manage virtual machines, hiding the intricacies of direct communication with KVM and QEMU, and providing portability across virtualization technologies. Libvirt has comprehensive support for operations such as creating VMs, managing the VM lifecycle (start, stop, delete), assigning resources, and monitoring, so it is best suited to use when creating an easy-to-use hypervisor.

In this architecture, Libvirt is employed to access the KVM hypervisor via the "qemu:///system" URI, establishing a long-lived connection which enables the hypervisor to send commands to the virtualization layer. Libvirt creates XML configurations that specify the attributes of each VM, such as its name, memory allocation, number of vCPUs, path of disk image, ISO file for boot, network setup, and graphical configuration. For instance, the XML configuration sets the VirtIO disk as the main storage device, VirtIO network interface for networking, and VNC graphics device for remote graphical access through TigerVNC. Libvirt also applies resource limits and isolation policies such that each VM runs within the assigned resources and cannot conflict with the host or other VMs. By offering a top-level abstraction, Libvirt abstracts away the complexity of doing virtualization tasks, leaving the hypervisor free to handle user interaction and logic and passing on low-level work to KVM and QEMU.

4.1.3 Remote Access Component

The remote access component is an essential aspect of the architecture, allowing users to access the graphical user interfaces of virtual machines across a network link.

4.1.3.1 TigerVNC:

TigerVNC is built into the architecture to enable remote graphical access to virtual machine consoles, so that users can use guest operating systems as if they were installed on a local system. TigerVNC is a VNC server-client system, in which the VNC server is implemented in the VM's setup, and the user accesses it by connecting through a VNC client. In this configuration, QEMU assigns each VM a VNC graphics device, with a port (e.g., autoport with listen address 0.0.0.0) that an automatically assigned is used to serve the VM's graphical output by TigerVNC.
TigerVNC is capable of high-performance remote desktop interaction, with capabilities like low-latency rendering and efficient compression, and can be used to interact with graphical guest OSes like Linux desktop environments or Windows. For Windows guests, which usually need graphical interaction for installation and setup, TigerVNC provides the ability for users to remotely access the VM's desktop environment, allowing them to perform activities like installing VirtIO drivers, setting up network configurations, or executing graphical applications. The inclusion of TigerVNC makes the hypervisor more user-friendly by providing access from a network remotely, which is especially useful in academic environments where users might access VMs from various locations.

4.1.4 Application Layer

The application layer encapsulates the custom logic and user-facing elements of the hypervisor, linking the underlying infrastructure together in order to provide an operational virtualization solution.

4.1.4.1 Hypervisor Implementation:

The hypervisor implementation forms the application layer, in which custom logic is created to consolidate KVM, QEMU, Libvirt, and TigerVNC into an integrated system. This layer consists of the user interface (CLI and GUI, according to the scope of the project) and the control systems for controlling virtual machines. The implementation prescribes the VM lifecycle operation workflow, including starting a new VM, starting a running VM, stopping or deleting a VM, and listing all VMs. It communicates with Libvirt so that it can pass on commands to the virtualization layer, to translate user inputs into functional virtualization operations. And, the hypervisor implementation also handles the resources allocation and validation to ensure the host system has enough resource (memory, CPU, disk space) to support the requested VMs. For example, when a VM is being created the implementation will check on user input of memory, vCPU's and the disk size to ensure that they are within acceptable limits before it proceeds with the creation of disk image and VM definition. It also manages
VirtIO device configuration for Windows guests such that when the VM is booted it is configured with the right drivers to provide it with best I/O performance. Application layer provides the feedback channel for the user so that a success message or error message is displayed for each of the operations successful or not such as VM creation success or startup status or deletion completion, in improving the user experience and debugging.

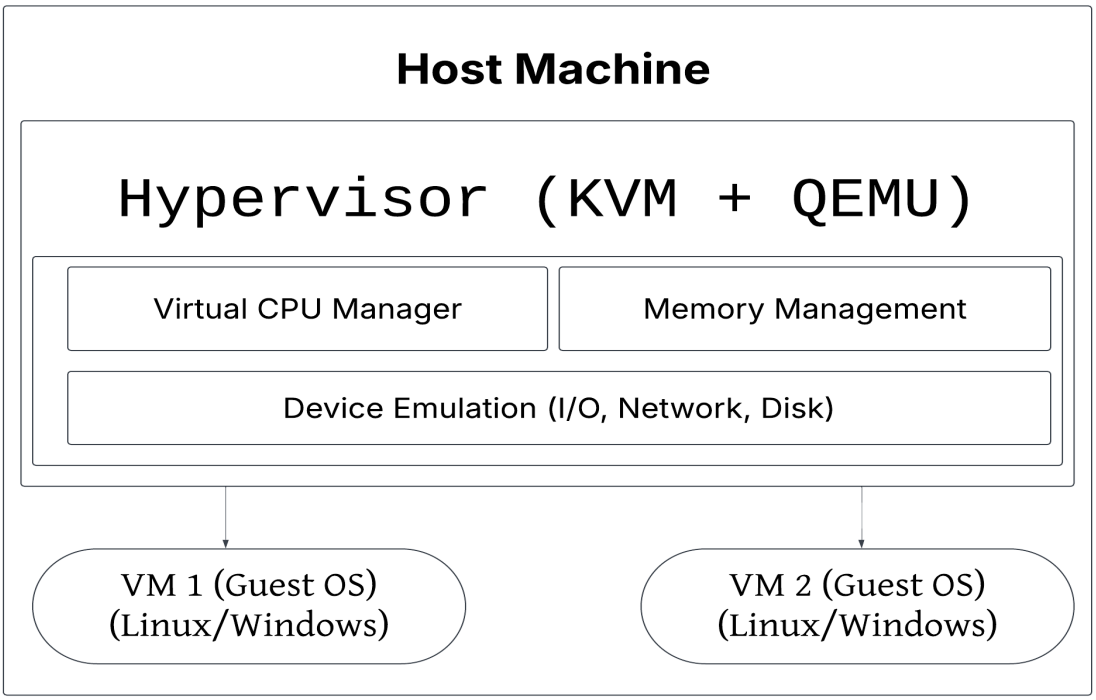


Fig 4.1.0 Overview of Project Architecture

4.2 WORKING OF THE PROJECT

The application layer entails the user-facing and the custom logic components of the hypervisor, which has the binding to the underlying infrastructure to create a working virtualization solution. The hypervisor does its job on an elaborate multi-layered process which handles communication between various virtualization components in order to offer proper virtual machine management. This complex process involves several stages, including ones that involve specific operations and interactions between the components that allow achieving the full lifecycle management of virtual machines.

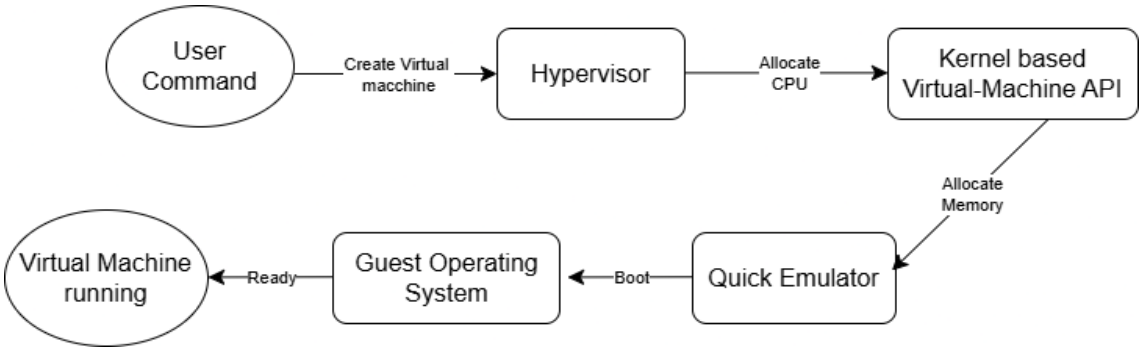


Fig 4.2.0 Workflow for VM Creation

4.2.1. Initialization

The initial boot process is the first call for the actions performed by hypervisors. At this initial phase, the hypervisor develops a secure conversation method to the base KVM (Kernel-based Virtual Machine environment with generic API of libvirt. This relationship is employed as the main channel through which all are passed. the rest of the virtualization commands and status enquiries.

The establishment of a connection uses the libvirt function virConnectOpen with the URI parameter "qemu:///system" to specify the system-level QEMU/KVM hypervisor. The approach to connection also demands proper system privileges so that only authorized entities can control virtualization resources. After establishing the connection, the hypervisor will initialize internal data structures to track state information concerning defined and active virtual machines.

Hardware virtualization support verification is also part of the initialization process, making sure that the host CPU supports the necessary extensions (Intel VT-x or AMD-V) to enable effective virtualization. The hypervisor asks the system capabilities via libvirt to ensure that the required virtualization features are present and enabled in the system BIOS/UEFI.

At this stage, the hypervisor also sets up interaction with the QEMU component, which would manage device emulation for virtual machines. The multi-component initialization guarantees proper provision of all required virtualization services before offering management choices to users.

4.2.2. Virtual Machine Creation

The creation of the VM is a complex set of actions that transmute user requirements into fully configured instances of virtual machines. The process begins with thorough collection of parameter including, VM name (unique ID), media path for installation(typically this is ISO), memory allocation, number of virtual CPU, the disk size.

When the hypervisor gets these parameters then it does strict validation about availability of resource and correctness of the parameters respectively. It verifies the existence and accessibility of the specified ISO file, as well as checking whether sufficient number of host resources are available to accommodate the requested memory and CPU allocation and ensures name conflicts with running VMs are not violated.

Disk image creation is one of the major operations which take place during this stage. The hypervisor deploys QEMU disk management function (an application for virtualization) to generate a qcow2 (QEMU Copy-On-Write version 2) format disk image. This format has numerous benefits, namely sparse allocation (the storage of only used blocks), snapshots, and

support for AES encryption. The disk create operation specifies the format and size which reserves the asked for capacity on host filesystem.After disk preparation, the hypervisor creates a complex XML configuration file that fully describes all facets of the virtual machine. This detailed specification contains memory allocation parameters (with room for memory ballooning), virtual CPU topology (cores, threads, and sockets), disk controller configurations, network interface settings, boot device order, and display options. For graphical access, the XML configuration includes VNC server parameters such as network binding options and auto-port allocation, supporting TigerVNC connections for remote graphical interaction with the virtual machine.
XML configuration also defines emulation parameters for the device, specifying how QEMU will expose virtualized hardware to the guest OS. These are video card emulation (usually Cirrus Logic or VGA compatible), input devices (keyboard, mouse), and controllers for different buses (IDE, SCSI, virtio). Advanced features like para-virtualized drivers for higher performance may also be included in the configuration.

4.2.3. Virtual Machine Execution

The VM run phase converts idle VM definitions into active virtual machines by a precisely coordinated sequence of operations by various virtualization components. When a user requests VM running, the hypervisor initially finds the domain object by the virDomainLookupByName function, which gets the VM configuration from its distinct name identifier.

Once it has retrieved the domain object successfully, the hypervisor activates the domain through the call of virDomainCreate. This operation invokes a highly intricate series of operations within the virtualization stack. KVM allocates and initializes the memory request of allocation, setting memory mapping between the host physical memory and guest virtual memory. It uses Extended Page Tables (EPT) or Nested Page Tables (NPT), hardware mechanisms that improve virtual-to-physical address translation speed and decrease virtualization overhead.
At the same time, KVM sets up virtual CPU instances (vCPUs) based on the VM configuration. Each vCPU is mapped to a host CPU core or scheduled over several cores depending on the scheduling policy of the host. KVM uses the CPU's virtualization extensions to create VMX (Virtual Machine Extensions) or SVM (Secure Virtual Machine) contexts, which allow guest code to be executed efficiently with little intervention.
While KVM sets up the virtualization environment, QEMU sets up device emulation according to the VM setup. This encompasses the setting up of virtual disk controllers that attach to the already set-up disk image, virtual network interfaces connecting to the defined network source (usually a virtual bridge), and input/output devices like virtual USB controllers, serial ports, and the graphical subsystem.

For graphical interaction, the hypervisor spawns a VNC server instance that is controlled by QEMU. The server listens on the given network interface (usually all interfaces through 0.0.0.0) and chooses a free port for it, enabling TigerVNC clients to remotely connect. The VNC server renders the virtual display adapter's framebuffer in real-time, delivering the graphical output to connected clients and proxying keyboard and mouse inputs back to the VM.
During the process of execution, KVM has complete isolation between the host system and the virtual machine. This isolation is also extended to memory spaces so that the VM cannot access memory outside its assigned areas, and to execution contexts so that privileged operations within the VM cannot influence the host system. This strong isolation is the core of the security model of KVM-based virtualization. After execution starts, the hypervisor tracks the VM's status and resource usage, offering management features during the VM's execution time. This includes the ability to dynamically change allocation of resources, snapshot, migrate VM to other hosts and manage it through power state in a range of control operations.

4.2.4. Virtual Machine Management

VM management is the step which involves the wide variety of activities that are aimed at controlling running VMs and checking their status. The hypervisor has improved enumeration capabilities boasting detailed view of active as well as idle virtual machines. Enumeration operation makes use of two varieties of libvirt functions: virConnectListDomains, a GLIST2 method that queries currently active VMs according to their numeric IDs, and virConnectListDefinedDomains, which queries VM definition that are not active by name. For each VM that was discovered the hypervisor obtains high-quality status information ie Resource utilization statistics, Run time in minutes and its current state (Running, paused, or terminated). In the case of running VMs the hypervisor provides a set of shutdown modes of different interference level. The courteous shut-down function, forced with virDomainShutdown, notifies guest operating system to be ready for normal shutdown processes. This technique is what allows the guest OS to gracefully kill processes, synchronize filesystems and perform other clean up functions before turning off. The hypervisor monitors the shutdown process, and updates the status in progress as the VM goes through the various stages of shutdown. For the case of emergencies requiring immediate shut down, forced termination is performed by the hypervisor with vir-DomainDestroy. This instantly stops VM execution without any involvement from the guest OS, equivalent of yanking power off a physical computer. Though useful, this method bypasses guest OS shutdown protocols and may lead to loss of data in case guest was at the middle of important operation. The hypervisor explicitly tells the users about these effects when forced termination is specified.

Apart from elementary power management, the stage of VM management comprehends advanced monitoring capabilities. The hypervisor can track resource utilization statistics, for example, CPU utilization, memory usage, disk I/O levels, and network transfer levels. Statistics of such a kind are good predictors of VM performance and performance demands of its resources, thus helping administrators in tuning the allocation of resources and identification of potential bottlenecks in the system. For the operations of deleting VM, a hypervisor has a very thorough cleanup mechanism that ensures all resources associated with VM are completely removed. This begins with checking the situation of the VM and stopping it now that it is running forcefully. The hypervisor then scans through the VM's XML setup to locate every resource it has, that is, the disk image file. After the undefined VM is deleted by virDomainUndefine as the secret to deleting its configuration from libvirt's database, the hypervisor deletes the corresponding disk image file such that there are no orphaned storage. Such a thorough cleaning process prevents resource leakage and maintains integrity of the system for long times of operations.

4.2.5. Isolation and Security

Security and isolation characteristics of the hypervisor are critical architectural building blocks, which guarantee safe functioning in multi-tenant deployments. KVM imposes serious memory isolation techniques, which allocation the physical memory among virtual machines, stricter way. The isolation takes advantage of hardware-assisted virtualization extensions in particular Second Level Address Translation (SLAT) technologies including Intel's Extended Page Tables (EPT) or AMD's Nested Page Tables (NPT). These advanced memory management technologies have separate page tables for all the virtual machines so that memory address in one VM cannot refer to the memory of another VM or the host. These page tables are initialized by the hypervisor during the VM startup and are maintained very well by the hypervisor when a VM is being operated in order to avoid illegal memory accesses on the VM boundaries. This separation of memories is the foundation for VM security, so that not even a compromised VM has the access to other virtual machine data. Apart from the memory isolation, KVM also follows strong CPU isolation by employing hardware virtualization extensions. All the virtual CPUs run inside a protected execution context, and KVM traps and emulates sensitive operations. This guarantees that the guest operating systems cannot use the privileged instructions which could compromise the host system and other VMs. The hypervisor takes good care for these execution contexts and ensures that there is clean separation of machine resources between different virtual machines. Libvirt applies this isolation further with strong access control mechanisms that monitor assignment and oversight operations of resources. These controls have the effect of making sure that users have just the required permissions for operation on operations that affect VM configurations or status to be carried out. The hypervisor is coupled with these access control

systems and requires call for permissions for such operations as starting, shutting-down or modifying VMs.

For isolation of storage, the hypervisor employs file-level access controls and certain area for storage in order to prevent access to unauthorized disk images of VM. All disk images are located in one managed directory (/var/lib/libvirt/images/) with restricted access in order to prevent unprivileged users from reading those directly or tampering with VM storage. Encryption features can also be used for encrypting VM data at rest for more protection in the qcow2 disk format. Network isolation is another important security feature. The Hypervisor provides virtual network interface, which is associated with isolated virtual networks or bridges to prevent unauthorized network traffic between VMs unless authorized. This kind of network isolation can implement such advanced security policies as traffic filtering, bandwidth limit, and network address translation. In order to provide a graphical access via TigerVNC, the hypervisor employs authentication mechanisms in order to prevent illegal connections to the VM console. These can take the form of password authentication, IP based access control as well as TLS encryption to provide safe remote access. Since hypervisor handles such authentication settings with caution there are only proper users who can view or even interact with the VM graphical interfaces. This broader scope of isolation and security limits that run across memory, CPU, storage, network and access control domains, the hypervisor provides a strong security stance that protects host and guest systems from various attack lines and unlawful access opportunities.

This many-layer security model offers the guarantee of smooth deployment of numerous virtual machines at shared environments but with good isolation guarantees.

The hypervisor implementation forms the application layer, where application-specific logic is written to put together KVM, QEMU, Libvirt, and TigerVNC into an integrated system. This layer incorporates the user interface (CLI and GUI, according to the project scope) and the control mechanisms used to manage virtual machines. The implementation determines the flow of VM lifecycle operations, i.e., to create a new VM, to start an existing VM, to stop or delete a VM, or to list all VMs. It talks to Libvirt in order to command the virtualization layer and translates user inputs into executable virtualization tasks.

The hypervisor implementation also provides resource allocation and validation to ensure that there are enough resources (memory, CPU, disk space) available in the host system to host the requested VMs. For instance, in creating a VM, the implementation checks user input for memory, vCPUs, and disk size to ensure they are within proper limits before creating disk images and defining VMs. It also controls the configuration of VirtIO devices for Windows guests so that the VM can be configured with the right drivers to maximize I/O performance. The application layer gives a feedback loop to the user, indicating success or failure messages for each operation, such

as VM creation confirmation, startup status, or deletion completion, to improve user experience and allow for debugging.

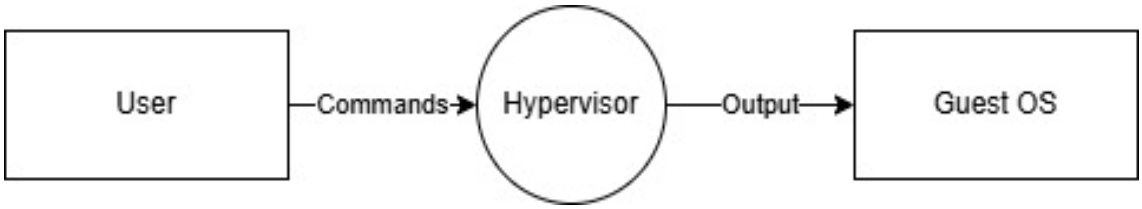


Fig 4.2.4 Data Flow Diagram - Level 0

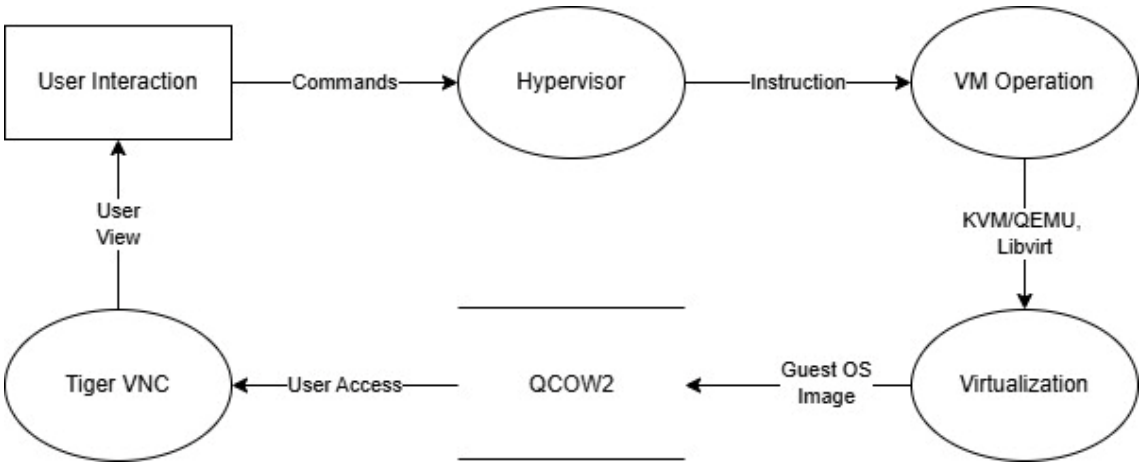


Fig 4.2.5 Data Flow Diagram - Level 1

4.3 FEATURES

4.3.1 Memory Management and Resource Allocation

The hypervisor design manages advanced resource and memory handling:

1. Dynamic Memory Allocation: Virtual machines are allocated fixed memory allocations given in KiB units (as observed in the XML configuration when memory is specified), and there is complete control over the distribution of resources.
2. Memory Overcommitment Protection: The system checks memory requests against available host resources to avoid overallocation situations that may cause system instability.

3. Virtual CPU Allocation: Multiple virtual CPUs can be assigned to each VM, with the hypervisor taking care of scheduling and running the vCPUs on physical CPU cores, managing context switching and state saving.
4. Disk Management: Virtual machines use QCOW2 (QEMU Copy-On-Write version 2) disk format, supporting thin provisioning, snapshots, and encryption features. The system handles disk creation, allocation, and deletion through organized paths at /var/lib/libvirt/images/.

4.3.2 VM Lifecycle Management

The hypervisor offers full VM lifecycle operations:

1. VM Creation Process:
    - Verifies the presence of the given ISO file
    - Creates a new QCOW2 disk image with the desired capacity
    - Builds detailed XML configuration specifying the VM's virtual hardware
    - Registers the VM with libvirt using the domain definition API
    - Configures boot order prioritization (CD-ROM then hard disk)
2. VM Startup Mechanism:
    - Finds the domain by name using domain lookup functions
    - Synchs boot process using virDomainCreate API
    - Sets up VNC connectivity for graphical access
    - Mem Allocates and maps memory pages to guest OS
    - Virtual CPU state initialization
3. VM Shutdown Procedures:
    - Provides both graceful shutdown (virDomainShutdown) which sends ACPI signals for guest OS cleanup
    - Offers forced termination (virDomainDestroy) for hung VMs
    - Cleans up resource deallocation and state cleanup
4. VM Deletion Process:
    - Powers off VM prior to deletion
    - Resolves disk path info from domain XML
    - Undefines domain config using virDomainUndefine
    - Physically deletes related disk files to avoid orphaned storage

4.3.3 Domain Management and Isolation

The system has strong domain management:

1. Domain Tracking:
    - Has independent tracking of active and inactive domains
    - Associates domain IDs with domain names for active VMs
    - Has domain enumeration via virConnectListDomains and virConnectListDefinedDomains
2. Security Isolation:
    - Uses KVM's hardware-based virtual machine isolation
    - Has independent memory spaces for every VM
    - Has network isolation via the default virtual network
    - Manages resource access via virtual device permissions
3. XML-Based Configuration:
    - Describes domains in terms of standardized libvirt XML schema
    - Supports full hardware specification with memory, CPU, storage, and networking
    - Configures graphics access via VNC with automatically assigned ports
    - Sets correct device bus types (virtio for disks, IDE for CD-ROM)

4.3.4 Networking and External Access

The installed hypervisor manages networking and outside access:

1. Network Setup:
    - Takes advantage of the default libvirt network for VM connections
    - Uses the virtio model of networks for enhanced performance
    - Supports NAT-based outgoing connectivity for VMs
2. Remote Access Integration:
    - Set up VNC to listen on all interfaces (0.0.0.0)
    - Executes automatic port assignment to avoid conflicts
    - Supports remote graphical access for VM administration and interaction
3. VIRTIO Integration:
    - Deploys paravirtualized storage with virtio bus for base disks
    - Utilizes classic IDE bus for CD-ROM drives for maximum compatibility during OS installation
    - Optimizes I/O performance via direct communication channels between guest and host

4.3.5 Advanced Features and Error Handling

The hypervisor has advanced error management and features:

1. Extensive Error Handling:
    - Verifies all API calls with proper error checking
    - Gives detailed error messages for operations that fail
    - Does resource cleanup to avoid memory leaks
2. Resource Verification:
    - Verifies ISO file presence prior to VM creation
    - Verifies VM presence prior to performing operations
    - Verifies VM state prior to trying state transitions
3. Disk Image Management:
    - Creates disk images in efficient qcow2 format
    - Does virtual disk cleanup during VM removal
    - Avoids corruption due to correct file handling

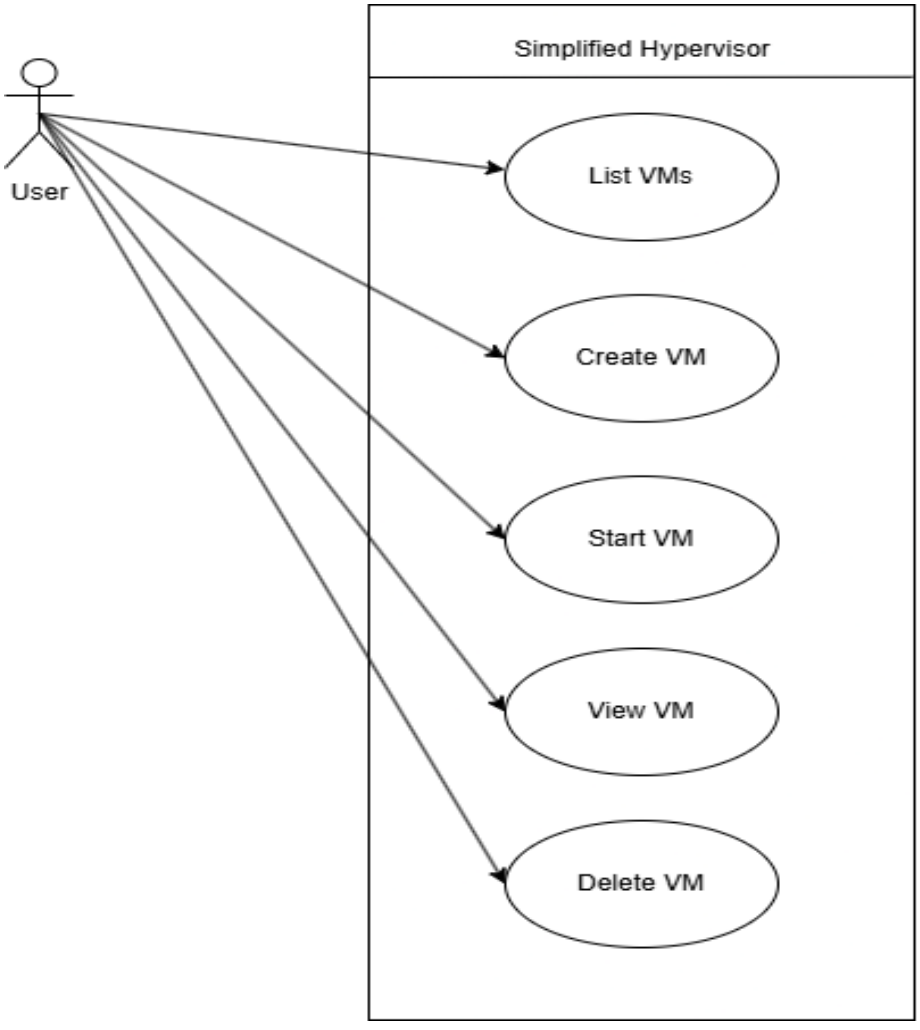


Fig 4.3.0 Use Cases of Hypervisor

CHAPTER - 5
RESULTS

5.1 HARDWARE AND SOFTWARE CONFIGURATION

Test Environment 1

- CPU: Intel Core i5-12400F (2.50 GHz, 6 cores, 12 threads)
- RAM: 16GB DDR5 (5200 MHz)
- Storage: 1TB HDD
- GPU: NVIDIA RTX 4060
- OS: Linux (Ubuntu – Host OS)

Test Environment 2

- CPU: AMD Ryzen 5 5500U (2.10 GHz, 6 cores, 12 threads)
- RAM: 12GB DDR4 (2667 MHz)
- Storage: 120GB SSD
- GPU: Integrated Radeon Graphics
- OS: Linux (Ubuntu – Host OS)

Software Stack:

- Hypervisor Framework: KVM (Kernel-based Virtual Machine)
- VM Management: QEMU and libvirt
- Remote Display: TigerVNC
- Windows Guest Support: VirtIO drivers

5.2 PROJECT OUTCOMES

VM Management Capabilities

The developed hypervisor successfully demonstrated the following capabilities across both test environments:

1. VM Creation and Deployment
    - Successfully created virtual machines with configurable parameters:
        - Memory allocation (from 1GB to 8GB tested)

        - vCPU allocation (from 1 to 4 cores tested)
        - Storage allocation (from 10GB to 100GB tested)
    - Supported direct boot from ISO files for OS installation
2. VM Lifecycle Management
    - Reliable VM startup and shutdown operations
    - State persistence between host system reboots
    - Graceful and forced termination capabilities
3. Guest OS Support
    - Successfully installed and ran multiple Linux distributions
    - Successfully installed and ran Windows with VirtIO drivers
    - Observed proper hardware resource allocation

5.2.1 Command Line Interface (CLI) Approach:

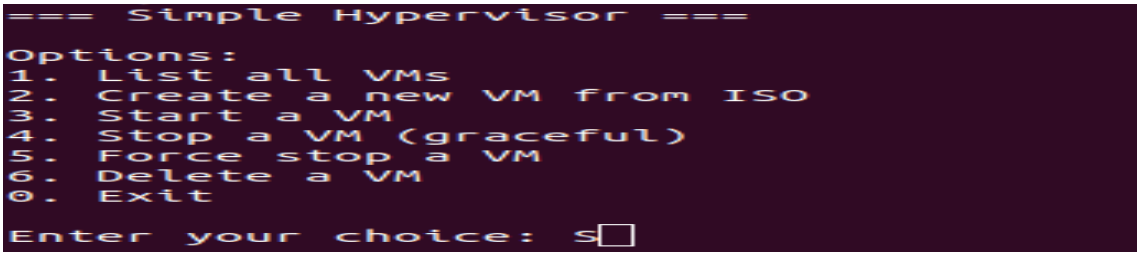


Fig 5.2.1.1 CLI Based Hypervisor Options



Fig 5.2.1.2 Creating a VM Using CLI

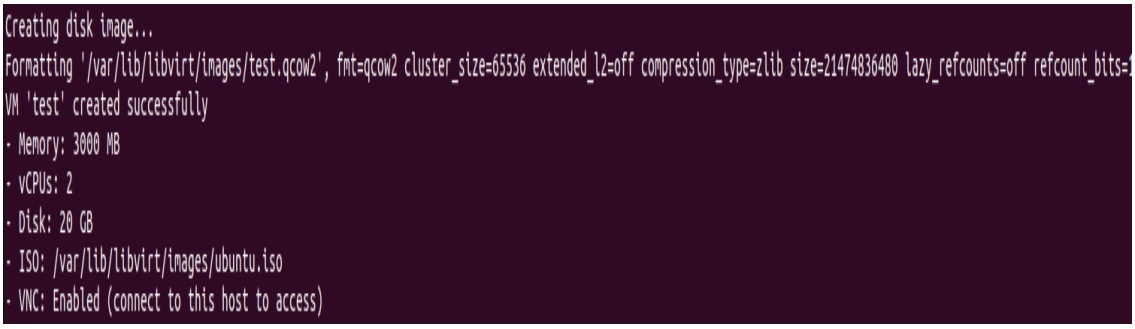


Fig 5.2.1.3 VM Creating Successful

The command-line interface delivered all management functionality with minimal complexity:

- VM creation took only 5 user inputs
- VM management tasks (start, stop, delete) took only 1-2 user inputs
- TigerVNC integration facilitated easy graphical access to guest systems

5.2.2 Graphical User Interface (GUI) Approach:

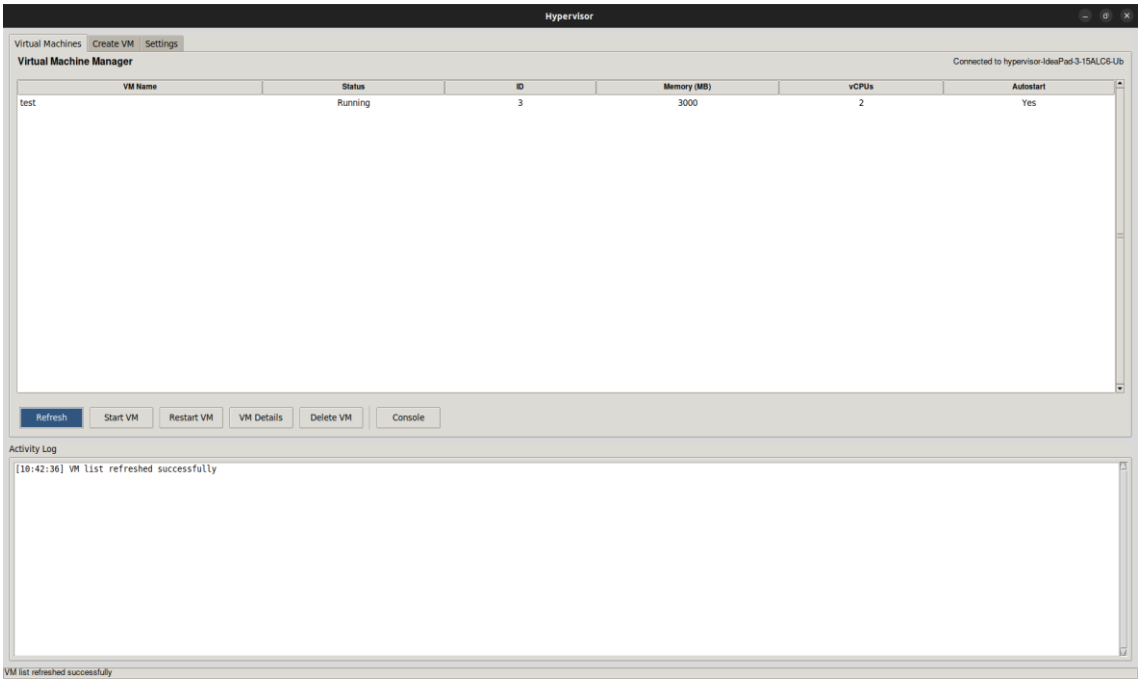


Fig 5.2.1. VM Management

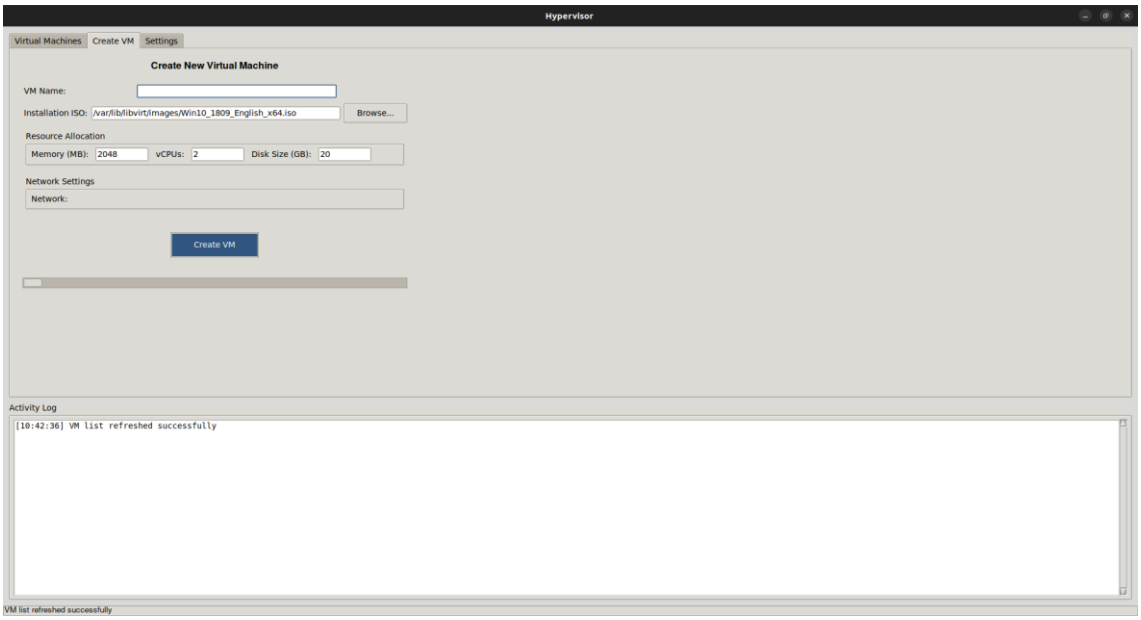


Fig 5.2.2 Create VM Configuration Interface

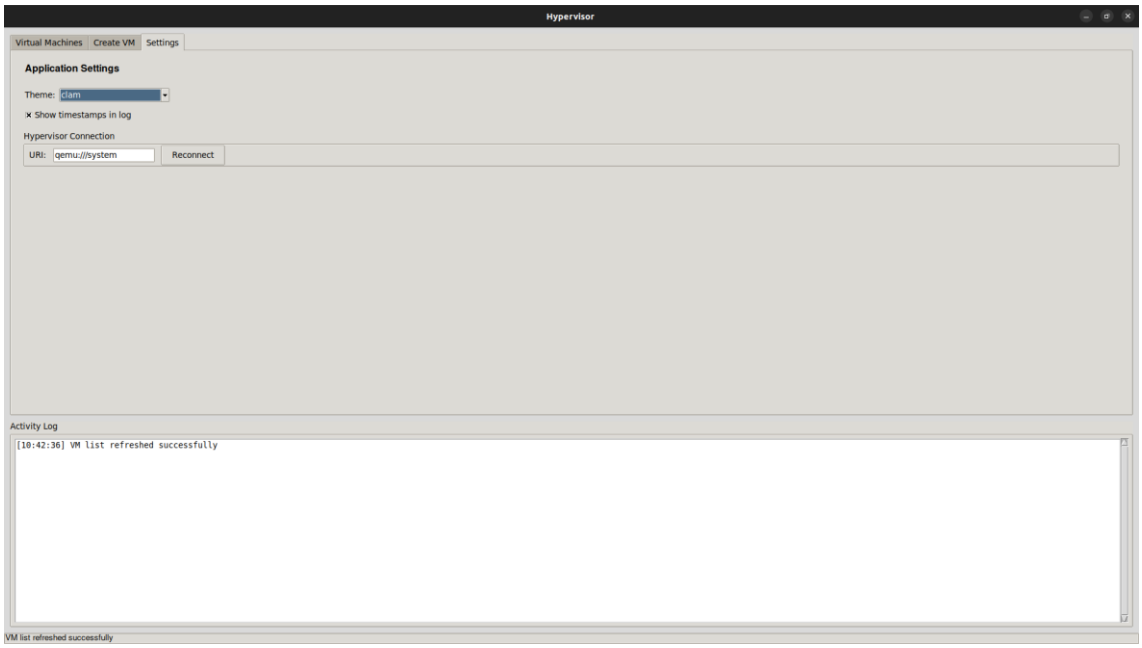


Fig 5.2.3 Hypervisor Settings Panel

Tkinter-based GUI has been able to convert the command-line operation into a simple visual management system:

- The tabbed interface structures functionality sensibly into "Virtual Machines," "Create VM," and "Settings" sections
- Form-based VM creation makes it easier with neatly labeled input fields for critical parameters
- Resource allocation sliders and input fields give feedback on config choices visually
- The bottom activity log gives real-time feedback about system operations

5.3 PERFORMANCE ANALYSIS

Table 5.3.1 Resource Utilization Comparison

Resource	**Intel Platform**	**AMD Platform**	**Notes**
CPU Overhead	3-5%	5-8%	Higher overhead on AMD due to lower base frequency
Memory Overhead	~200MB per VM	~200MB per VM	Consistent across platforms
Disk I/O Performance	Excellent	Very Good	SSD on AMD platform showed better I/O despite lower CPU performance

Network Throughput	850-950 Mbps	800-900 Mbps	Tested with iperf3 between host and guest

Table 5.3.2 VM Boot Time Analysis

Guest OS	**Intel Platform**	**AMD Platform**
Ubuntu 22.04	15s	13s
Kali Linux	14s	14s
Windows 10	28s	25s

Testing multiple concurrent VMs showed linear resource scaling up to the hardware limits:

- Intel platform supported up to 5 concurrent VMs with 2GB RAM each before performance degradation
- AMD platform supported up to 3 concurrent VMs with 2GB RAM each before performance degradation

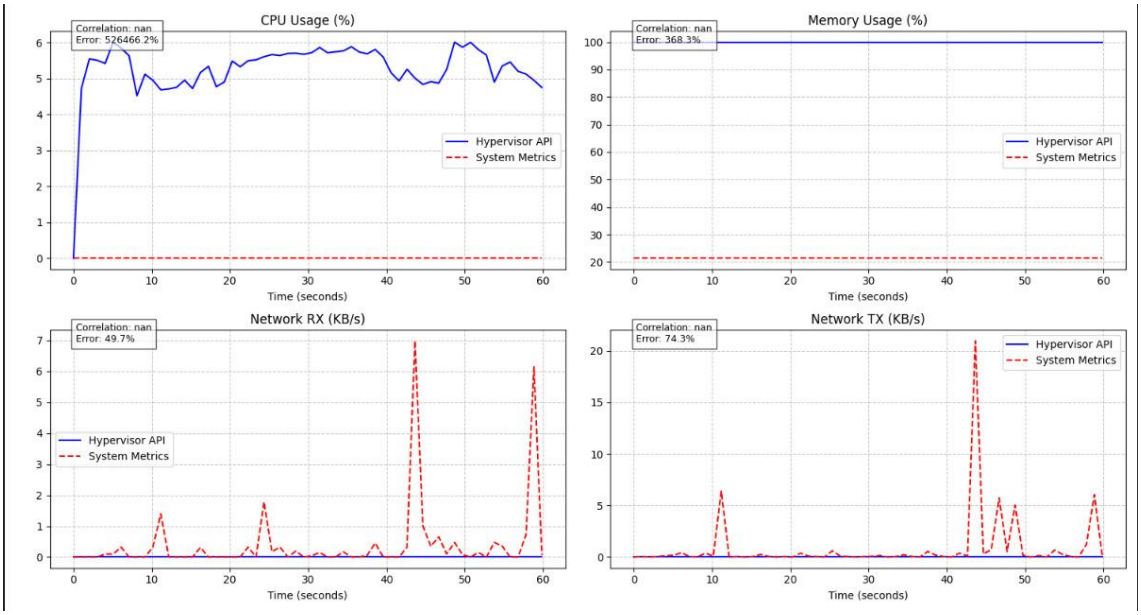


Fig 5.3.1 Hypervisor vs System (Performance Metrices)

5.3.1 Resource & Permission Management

Permission Management

- Early issues with disk image creation permissions were addressed by executing with elevated privilege or modifying directory permissions
- Accurate group membership setup allowed for non-root operation

Driver Compatibility

- VirtIO drivers had to be installed on Windows guests for installation & best performance
- Solution adopted: VirtIO driver integration at the time of VM creation

Disk Space Management:

- QCOW2 thin provisioning resulted in only 30-40% actual usage of allocated space for new VMs
- Disk files were systematically and entirely deleted during VM delete operations

Connection Handling:

- libvirt connections were correctly set up and shut down
- Resource leak not found in connection handling

Memory Reclamation:

- 100% memory allocated was correctly reclaimed after deleting VM
- Optimal VM density per hardware setup found through testing

The hypervisor implementation effectively achieved its performance goals on various hardware configurations. The Intel system with faster clock speeds showed superior raw performance, whereas the AMD system with SSD storage offered quicker I/O operations.

The combination of KVM's hardware virtualization features and QEMU's device emulation capabilities led to a hypervisor solution that optimizes performance while ensuring educational openness. TigerVNC integration offered seamless graphical access to virtual machines, and VIRTIO drivers provided improved performance for supported guest operating systems.

CHAPTER - 6
CONCLUSION

6.1 CONCLUSION

This project aimed to create a working & lightweight hypervisor system that offers in-depth insight into low-level virtualization mechanisms. In contrast to conventional black-box solutions like VMware or VirtualBox, which hide much of the underlying architecture, this implementation focuses on direct interaction with primary open-source virtualization technologies, namely KVM, QEMU, Libvirt, and TigerVNC, to provide both practical usability and conceptual insight. The main goal was to facilitate the creation, management, and interaction with virtual machines in a controlled, extensible, and performance-conscious environment while also encouraging a better academic understanding of how hypervisors work under the hood.

The approach took a layered architecture to represent the modularity and dependency of elements in contemporary virtualization stacks. KVM was the central virtualization module integrated into the Linux kernel and used hardware acceleration technologies like Intel VT-x and AMD-V to facilitate secure and isolated CPU virtualization. On top of KVM, QEMU managed device emulation and virtual hardware abstraction, enabling compatibility with a variety of guest operating systems. Libvirt was central to controlling virtual machine orchestration and lifecycle activities through a common API that made it easy to interact with the lower layers. To make graphical user access to virtual machines easy, TigerVNC was integrated to provide easy remote GUI access to Linux and Windows-based virtual machines. VirtIO drivers were also employed specifically for Windows guests to improve input/output speeds by providing paravirtualized interfaces for storage and networking.

The final outcome was a complete and functional Type-2 hypervisor developed entirely with open-source technology, showcasing the capacity to install and operate multiple virtual machines with discrete resource distribution in a manner that preserves system stability and performance. The VirtIO integration served to enhance input/output performance in Windows guests by a very high margin, hence making the solution not only instructive but also applicable in real-world settings. Remote access functionality using TigerVNC also increased the ease of use of the system, particularly in multi-guest operating system environments. Above all, the project achieved providing a transparent and hands-on teaching platform that graphically represents the

function of each virtualization level, such as CPU, memory, and I/O, thus enhancing conceptual learning as well as technical capability in system-level software development.

6.2 CONTRIBUTION TO SUSTAINABLE DEVELOPMENT GOALS (SDGS):

- SDG 9: Industry, Innovation, and Infrastructure – The hypervisor project contributes to technological innovation by virtualization capabilities, which are essential for cloud computing, IT infrastructure, and efficient resource utilization.
- SDG 12: Responsible Consumption and Production – By enabling efficient resource management through virtualization, the project reduces hardware dependency, leading to lower energy consumption and electronic waste.

6.3 LIMITATIONS

Despite its success, the system outlined in this paper has limitations compared to great and full-fledged virtualization platforms. Among other things, it lacks high-end enterprise features such as live migration, snapshotting automated scaling and high-availability clustering that are vital for fault-tolerant and large scale-deployments. Second, despite sufficing for Academic work and small scale operation using a Type-2 architecture incurs performance overhead that may compromise scalability in respect to high-resource settings or throngs of concurrent Virtual Machine's. Third, advanced security features such as encrypted storage for VMs, secure boot, role-based access control and run-time threat detection, which are mandatory when used in production environments of security-critical domains, are not present in the code. Furthermore, the system has no integration of cloud management and orchestration software like OpenStack or Kubernetes, and thus is not so suitable in the modern, automated infrastructure. Finally, though TigerVNC does support rudimentary GUI-style access to virtual machines, the lack of a dedicated, fully developed graphical interface for the management of hypervisors does make it a less appealing desktop system to more non-technical users and lowers the accessibility of the system outside of educational environments.

CHAPTER - 7
FUTURE SCOPE

The existing implementation of the Type-2 hypervisor provides a solid foundation for virtualization learning and lightweight virtual machine (VM) management, providing an open platform for learning and experimentation. Nonetheless, there are some areas of future improvement that can greatly enhance its functionality, performance, and applicability to a wide range of use cases, bringing it closer to academic and industry requirements.

Advanced Monitoring and Analytics for Enhanced Visibility

Merging real-time monitoring and analytics would yield useful information about VM resource utilization (CPU, memory, disk I/O, network). Prometheus and Grafana could provide visualization, allowing proactive resource optimization and bottleneck detection. Automated benchmarking scripts with user-configurable sampling intervals (e.g., 1 second to 1 minute) could profile performance under different workloads, recording boot times, latency, and throughput for thorough analysis and optimization.

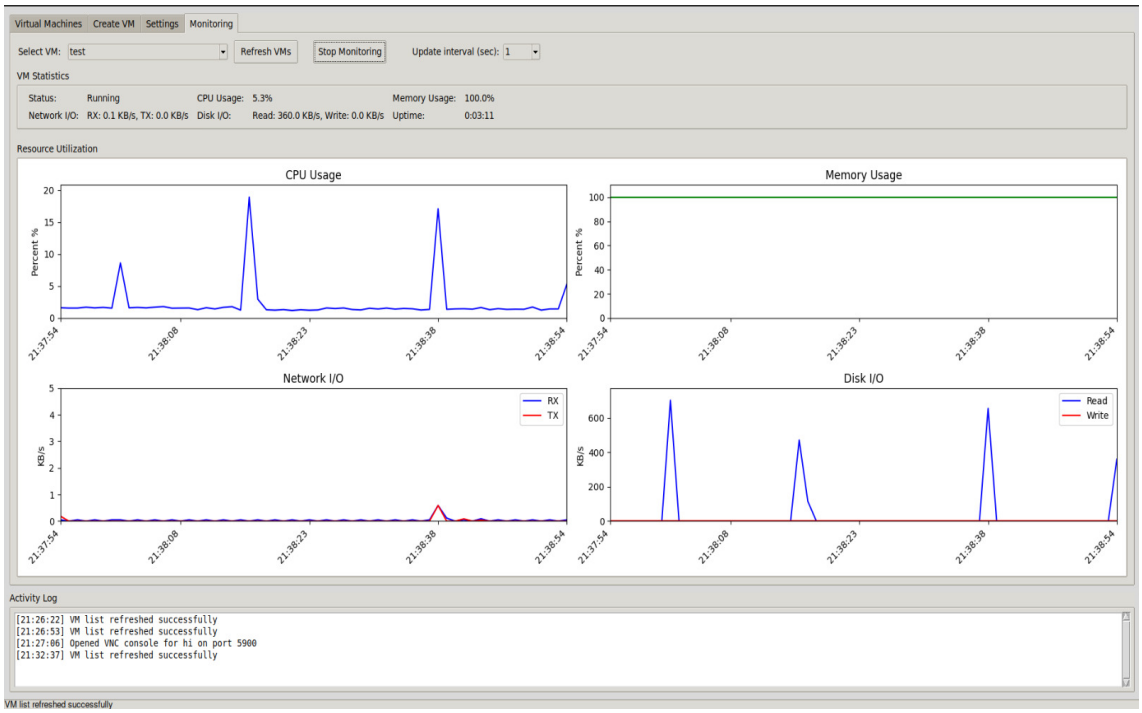


Fig 7.1.0 Prototype for Monitoring

Robust Error Handling and Improved Reliability

Error handling needs to be improved. Complete logging frameworks (such as syslog or a module of your choice) would trap rich error traces and system events. Automated recovery mechanisms, such as VM resets and resource redistribution on failure, would enhance reliability. In addition, fault isolation methods, such as sandboxing VM processes, would avoid cascading failures and provide higher system stability and availability.

Integration of Advanced Virtualization Features

Long-term growth would center on bringing together advanced virtualization capabilities that are present in enterprise solutions. Live migration of zero-downtime VM migration, VM snapshotting for saving and restoring the state, and secure storage implementations (such as LUKS encryption) would add immense usefulness to it. Delving into cloud orchestration integration with such frameworks as OpenStack or Kubernetes could also put the hypervisor on the map for scalable cloud infrastructures.

Optimizing I/O Performance for Graphical Workloads

Future work may improve I/O performance for graphics workloads. Expanding VirtIO to encompass VirtIO-GPU would provide hardware-accelerated graphics rendering for Linux (e.g., GNOME, KDE) and Windows guests, useful for educational environments with graphical applications. Adding nested virtualization support would also make it possible for sophisticated use cases such as testing Type-1 hypervisors or running container runtimes inside VMs.

REFERNCES

Research Papers:

[1] J. Szefer and R. B. Lee, "Architectural support for hypervisor-secure virtualization," ACM SIGPLAN Notices, vol. 47, no. 4, pp. 437–450, Jun. 2012, doi: https://doi.org/10.1145/2248487.2151022.

[2] J. Yang and K. G. Shin, "Using hypervisor to provide data secrecy for user applications on a per-page basis," Virtual Execution Environments, Mar. 2008, doi: https://doi.org/10.1145/1346256.1346267.

[3] Z. Aalam, V. Kumar, and S. Gour, "A review paper on hypervisor and virtual machine security," Journal of Physics: Conference Series, vol. 1950, no. 1, p. 012027, Aug. 2021, doi: https://doi.org/10.1088/1742-6596/1950/1/012027.

[4] Ammarit Thongthua and Sudsanguan Ngamsuriyaroj, "Assessment of Hypervisor Vulnerabilities," May 2016, doi: https://doi.org/10.1109/icccri.2016.19.

[5] A. Seshadri, M. Luk, N. Qu, and A. Perrig, "SecVisor," Proceedings of twenty-first ACM SIGOPS symposium on Operating systems principles – SOSP '07, 2007, doi: https://doi.org/10.1145/1294261.1294294.

[6] M. Kiperberg, R. Leon, A. Resh, Asaf Algawi, and N. J. Zaidenberg, "Hypervisor-Based Protection of Code," IEEE Transactions on Information Forensics and Security, vol. 14, no. 8, pp. 2203–2216, Jan. 2019, doi: https://doi.org/10.1109/tifs.2019.2894577.

[7] T. C. Bressoud and F. B. Schneider, "Hypervisor-based fault tolerance," ACM Transactions on Computer Systems, vol. 14, no. 1, pp. 80–107, Feb. 1996, doi: https://doi.org/10.1145/225535.225538.

Book:

"Virtual Machines: Versatile Platforms for Systems and Processes" by Jim Smith and Ravi Nair (2005)
Chapter 1: Introduction to Virtual Machines
Topic: Covers basics of virtualization and hypervisor architectures relevant to the Type-2 hypervisor design used in the project.

Articles:

Kernel-based Virtual Machine (KVM)
https://www.linux-kvm.org/page/Documents

Quick Emulator (QEMU)
https://www.qemu.org/docs/master/

libvirt
https://libvirt.org/docs.html

TigerVNC
https://tigervnc.org/
https://github.com/TigerVNC/tigervnc

Ubuntu
https://help.ubuntu.com/

APPENDIX 1

HYPERVISOR.CPP (CLI BASED):

```
#include <iostream>
#include <string>
#include <vector>
#include <libvirt/libvirt.h>
#include <memory>
#include <fstream>
#include <unistd.h>
using namespace std;
/** Lists all VMs (active and inactive) connected to the hypervisor */
void listAllVMs(virConnectPtr conn) {
    int numActive = virConnectNumOfDomains(conn);
    vector<int> activeDomains(numActive);
    virConnectListDomains(conn, activeDomains.data(), numActive);
    int numInactive = virConnectNumOfDefinedDomains(conn);
    vector<char*> inactiveDomains(numInactive);
    virConnectListDefinedDomains(conn, inactiveDomains.data(), numInactive);
    cout << "===== ACTIVE VMs =====" << endl;
    for (int i = 0; i < numActive; i++) {
        virDomainPtr dom = virDomainLookupByID(conn, activeDomains[i]);
        if (dom) {
            cout << "- " << virDomainGetName(dom) << " (ID: " << activeDomains[i] << ")" << endl;
            virDomainFree(dom); } }
    cout << "===== INACTIVE VMs =====" << endl;
    for (int i = 0; i < numInactive; i++) {
        cout << "- " << inactiveDomains[i] << " (Inactive)" << endl;
        free(inactiveDomains[i]); } }
/** Starts a virtual machine by name */
bool startVM(virConnectPtr conn, const char* name) {
    virDomainPtr dom = virDomainLookupByName(conn, name);
    if (!dom) {
```

```
        cerr << "VM '" << name << "' not found." << endl;
        return false; }
    if (virDomainCreate(dom) < 0) {
        cerr << "Failed to start VM '" << name << "'." << endl;
        virDomainFree(dom);
        return false; }
    cout << "VM '" << name << "' started." << endl;
    virDomainFree(dom);
    return true; }
/** Performs graceful shutdown of a VM */
bool stopVM(virConnectPtr conn, const char* name) {
    virDomainPtr dom = virDomainLookupByName(conn, name);
    if (!dom) {
        cerr << "VM '" << name << "' not found." << endl;
        return false; }
    if (virDomainShutdown(dom) < 0) {
        cerr << "Failed to stop VM '" << name << "'." << endl;
        virDomainFree(dom);
        return false;   }
    cout << "VM '" << name << "' is shutting down..." << endl;
    virDomainFree(dom);
    return true; }
/** Forcefully stops a VM (equivalent to unplugging power) */
bool forceStopVM(virConnectPtr conn, const char* name) {
    virDomainPtr dom = virDomainLookupByName(conn, name);
    if (!dom) {
        cerr << "VM '" << name << "' not found." << endl;
        return false;   }
    if (virDomainDestroy(dom) < 0) {
        cerr << "Failed to force stop VM '" << name << "'." << endl;
        virDomainFree(dom);
        return false; }
    cout << "VM '" << name << "' forcibly stopped." << endl;
    virDomainFree(dom);
    return true; }
/** Creates a new virtual machine using an ISO and allocates resources */
```

```
bool createVM(virConnectPtr conn, const string& name, const string& iso, int memMB = 2048, int vcpus = 2, int diskGB = 20) {
    ifstream file(iso);
    if (!file.good()) {
        cerr << "ISO file not found: " << iso << endl;
        return false; }
    string diskPath = "/var/lib/libvirt/images/" + name + ".qcow2";
    string cmd = "qemu-img create -f qcow2 " + diskPath + " " + to_string(diskGB) + "G";
    if (system(cmd.c_str()) != 0) {
        cerr << "Disk creation failed." << endl;
        return false; }
    string xml = "<?xml version='1.0'?>\n"
// This XML defines the virtual machine configuration for libvirt.
// It includes VM name, memory, CPU, boot order, disk and ISO paths,
// network interface using default bridge, and VNC graphics.
// The VM uses KVM virtualization with VirtIO and ACPI/APIC support.
    virDomainPtr dom = virDomainDefineXML(conn, xml.c_str());
    if (!dom) {
        cerr << "VM definition failed." << endl;
        return false; }
    cout << "VM '" << name << "' created with " << memMB << "MB RAM, " << vcpus << " CPU(s), and " << diskGB << "GB disk." << endl;
    virDomainFree(dom);
    return true; }
<< name << "' deleted." << endl;
    return true; }
/** CLI menu entry point for the virtual machine manager */
int main() {
    virConnectPtr conn = virConnectOpen("qemu:///system");
    if (!conn) {
        cerr << "Failed to connect to hypervisor." << endl;
        return 1; }
    cout << "=== Simple VM Manager ===" << endl;
    bool running = true;
    while (running) {
        cout << "\n1. List VMs\n2. Create VM\n3. Start VM\n4. Stop VM\n5. Force Stop VM\n6. Delete VM\n0. Exit\nChoice: ";
```

```
        int choice;
        cin >> choice;
        cin.ignore();
        string name, iso;
        int mem, cpu, disk;
        switch (choice) {
            case 1: listAllVMs(conn); break;
            case 2:
                cout << "VM Name: "; getline(cin, name);
                cout << "ISO Path: "; getline(cin, iso);
                cout << "Memory (MB): "; cin >> mem;
                cout << "vCPUs: "; cin >> cpu;
                cout << "Disk (GB): "; cin >> disk;
                createVM(conn, name, iso, mem, cpu, disk); break;
            case 3: cout << "VM to Start: "; getline(cin, name); startVM(conn, name.c_str()); break;
            case 4: cout << "VM to Stop: "; getline(cin, name); stopVM(conn, name.c_str()); break;
            case 5: cout << "VM to Force Stop: "; getline(cin, name); forceStopVM(conn, name.c_str());
break;
            case 6: cout << "VM to Delete: "; getline(cin, name); deleteVM(conn, name.c_str()); break;
            case 0: running = false; break;
            default: cout << "Invalid option." << endl; } }
    virConnectClose(conn);
    return 0; }
```

HYPERVISOR.PY (GUI BASED):

```
import tkinter as tk
from tkinter import ttk, filedialog, messagebox, scrolledtext
import subprocess, os, libvirt, threading, time, re
from datetime import datetime
from slugify import slugify
import xml.etree.ElementTree as ET
class Tooltip:
    """Attaches a tooltip to any tkinter widget."""
    def __init__(self, widget, text):
        self.widget = widget
        self.text = text
```

```
        self.tip_window = None
        widget.bind('<Enter>', self.show_tip)
        widget.bind('<Leave>', self.hide_tip)
    def show_tip(self, _=None):
        x, y, *_ = self.widget.bbox("insert")
        x += self.widget.winfo_rootx() + 25
        y += self.widget.winfo_rooty() + 25
        self.tip_window = tk.Toplevel(self.widget)
        self.tip_window.wm_overrideredirect(True)
        self.tip_window.wm_geometry(f"+{x}+{y}")
        label = ttk.Label(self.tip_window, text=self.text, background="#ffffe0", relief=tk.SOLID,
borderwidth=1, padding=5)
        label.pack()
    def hide_tip(self, _=None):
        if self.tip_window:
            self.tip_window.destroy()
        self.tip_window = None
class EnhancedHypervisorManagerGUI:
    """GUI application for managing virtual machines using libvirt."""
    def __init__(self, root):
        self.root = root
        self.root.title("Hypervisor Manager")
        self.root.geometry("1000x700")
        self.conn = None
        self.style = ttk.Style()
        self.style.theme_use('clam')
        self.configure_styles()
        self.status_var = tk.StringVar(value="Ready")
        self.connection_status = tk.StringVar(value="Connecting...")
        ttk.Label(root, textvariable=self.status_var, relief=tk.SUNKEN, anchor=tk.W,
style='Status.TLabel').pack(side=tk.BOTTOM, fill=tk.X)
        self.connect_to_hypervisor()
        self.init_ui()
        self.refresh_vm_list()
    def configure_styles(self):
        """Configures custom styles for the widgets."""
        self.style.configure('TButton', padding=5)
```

```
        self.style.configure('Title.TLabel', font=('Helvetica', 12, 'bold'))
        self.style.configure('Status.TLabel', font=('Helvetica', 10))
        self.style.configure('Treeview', rowheight=25)
        self.style.configure('Treeview.Heading', font=('Helvetica', 9, 'bold'))
        self.style.map('Treeview', background=[('selected', '#347083')])
        self.style.configure('Error.TLabel', foreground='red')
        self.style.configure('Accent.TButton', foreground='white', background='#305680')
    def connect_to_hypervisor(self):
        """Connects to the system libvirt hypervisor in a background thread."""
        def thread_func():
            try:
                self.conn = libvirt.open("qemu:///system")
                if self.conn:
                    self.root.after(0, lambda: self.connection_status.set(f"Connected to
{self.conn.getHostname()}"))
                else:
                    self.root.after(0, lambda: self.connection_status.set("Connection failed"))
            except libvirt.libvirtError:
                self.root.after(0, lambda: self.connection_status.set("Connection failed"))
        threading.Thread(target=thread_func).start()


    def init_ui(self):
        """Initializes main GUI components and tabs."""
        notebook = ttk.Notebook(self.root)
        notebook.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)
        vm_tab = ttk.Frame(notebook)
        notebook.add(vm_tab, text="VMs")
        self.setup_vm_list_tab(vm_tab)
        create_tab = ttk.Frame(notebook)
        notebook.add(create_tab, text="Create VM")
        self.setup_create_vm_tab(create_tab)
        log_frame = ttk.LabelFrame(self.root, text="Activity Log")
        log_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=(0, 10))
        self.console = scrolledtext.ScrolledText(log_frame, height=8, wrap=tk.WORD,
state=tk.DISABLED)
        self.console.pack(fill=tk.BOTH, expand=True, padx=5, pady=5)
        self.console.tag_config('error', foreground='red')
```

```
        self.log_to_console("Application started.")

    def setup_vm_list_tab(self, parent):

        """Sets up the tab displaying all virtual machines."""

        ttk.Label(parent, text="Virtual Machine Manager", style='Title.TLabel').pack(anchor=tk.W,
padx=10, pady=5)

        ttk.Label(parent, textvariable=self.connection_status).pack(anchor=tk.E, padx=10)

        self.vm_tree = ttk.Treeview(parent, columns=('name', 'status', 'id', 'memory', 'vcpu', 'autostart'),
show='headings')

        for col in self.vm_tree['columns']:

            self.vm_tree.heading(col, text=col.title())

            self.vm_tree.column(col, width=100 if col != 'name' else 200, anchor=tk.CENTER)

        self.vm_tree.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

        self.vm_tree.bind('<Double-1>', self.show_vm_details)

        btn_frame = ttk.Frame(parent)

        btn_frame.pack(pady=5)

        actions = [("Refresh", self.refresh_vm_list), ("Restart", self.restart_vm), ("Details",
self.show_vm_details), ("Delete", self.delete_vm), ("Console", self.open_console)]

        for text, cmd in actions:

            ttk.Button(btn_frame, text=text, command=cmd).pack(side=tk.LEFT, padx=5)


    def setup_create_vm_tab(self, parent):

        """Sets up the tab for creating a new virtual machine."""

        ttk.Label(parent, text="Create New Virtual Machine", style='Title.TLabel').pack(pady=10)

        fields = ttk.Frame(parent)

        fields.pack(padx=20, pady=10, fill=tk.X)

        self.vm_name_entry = self._add_labeled_entry(fields, "VM Name:", 0)

        Tooltip(self.vm_name_entry, "Alphanumeric name with dashes/underscores")

        self.iso_path_entry = self._add_file_browser(fields, "Windows ISO:", 1, self.browse_iso)

        self.virtio_path_entry = self._add_file_browser(fields, "VirtIO ISO:", 2, self.browse_virtio)

        self.memory_entry = self._add_labeled_entry(fields, "Memory (MB):", 3, default="4096")

        self.vcpu_entry = self._add_labeled_entry(fields, "vCPUs:", 4, default="2")

        self.disk_entry = self._add_labeled_entry(fields, "Disk Size (GB):", 5, default="40")

        self.network_combo = ttk.Combobox(fields, state="readonly")

        self.network_combo.grid(row=6, column=1, sticky=tk.W)

        Tooltip(self.network_combo, "Choose the network bridge")

        self.load_available_networks()

        ttk.Label(fields, text="Network:").grid(row=6, column=0, sticky=tk.W)
```

```
        self.create_btn = ttk.Button(parent, text="Create VM", command=self.create_vm,
style='Accent.TButton')
        self.create_btn.pack(pady=10)
        self.progress_bar = ttk.Progressbar(parent, mode='indeterminate')
        self.progress_bar.pack(fill=tk.X, padx=20)
    def _add_labeled_entry(self, parent, label, row, default=""):
        """Helper to create a labeled entry."""
        ttk.Label(parent, text=label).grid(row=row, column=0, sticky=tk.W)
        entry = ttk.Entry(parent)
        entry.insert(0, default)
        entry.grid(row=row, column=1, sticky=tk.W)
        return entry
    def _add_file_browser(self, parent, label, row, command):
        """Helper to create file entry with a browse button."""
        ttk.Label(parent, text=label).grid(row=row, column=0, sticky=tk.W)
        entry = ttk.Entry(parent, width=40)
        entry.grid(row=row, column=1, sticky=tk.W)
        ttk.Button(parent, text="Browse...", command=command).grid(row=row, column=2, padx=5)
        return entry


    def browse_iso(self):
        """Selects a Windows ISO file."""
        path = filedialog.askopenfilename(filetypes=[("ISO files", "*.iso")])
        if path: self.iso_path_entry.delete(0, tk.END); self.iso_path_entry.insert(0, path)
    def browse_virtio(self):
        """Selects a VirtIO driver ISO."""
        path = filedialog.askopenfilename(filetypes=[("ISO files", "*.iso")])
        if path: self.virtio_path_entry.delete(0, tk.END); self.virtio_path_entry.insert(0, path)
    def load_available_networks(self):
        """Loads available virtual networks into the combo box."""
        try:
            networks = self.conn.listNetworks() + self.conn.listDefinedNetworks()
            self.network_combo['values'] = networks or ["default"]
            self.network_combo.set(networks[0] if networks else "default")
        except:
            self.network_combo['values'] = ["default"]
```

```
        self.network_combo.set("default")


    def log_to_console(self, message, error=False):
        """Logs a message to the console with optional error formatting."""
        self.console.config(state=tk.NORMAL)
        self.console.insert(tk.END, f"{datetime.now().strftime('%H:%M:%S')} - {message}\n", 'error' if
error else '')
        self.console.see(tk.END)
        self.console.config(state=tk.DISABLED)
        self.status_var.set(message[:100])
    def ensure_connection(self):
        """Checks if connection is active."""
        if not self.conn:
            messagebox.showerror("Error", "No connection to hypervisor.")
            return False
        return True


    def refresh_vm_list(self):
        """Refreshes the list of virtual machines."""
        if not self.ensure_connection(): return
        self.vm_tree.delete(*self.vm_tree.get_children())
        try:
            for dom_id in self.conn.listDomainsID():
                dom = self.conn.lookupByID(dom_id)
                self._insert_vm_row(dom, True)
            for name in self.conn.listDefinedDomains():
                dom = self.conn.lookupByName(name)
                self._insert_vm_row(dom, False)
            self.log_to_console("VM list refreshed.")
        except libvirt.libvirtError as e:
            self.log_to_console(f"Error refreshing list: {e}", error=True)
    def _insert_vm_row(self, dom, active):
        """Inserts a row into the tree view."""
        self.vm_tree.insert('', tk.END, values=(
            dom.name(), "Running" if active else "Stopped",
            dom.ID() if active else "-", dom.maxMemory() // 1024,
```

```
            dom.maxVcpus(), "Yes" if dom.autostart() else "No"))
    def show_vm_details(self, _=None):
        """Displays selected VM's details."""
        item = self.vm_tree.focus()
        if not item: return
        name = self.vm_tree.item(item)['values'][0]
        dom = self.conn.lookupByName(name)
        xml = ET.fromstring(dom.XMLDesc(0))
        arch = xml.find('./os/type').get('arch')
        info = f"Name: {name}\nUUID: {dom.UUIDString()}\nArch: {arch}"
        messagebox.showinfo("VM Details", info)
    def restart_vm(self):
        """Restarts the selected virtual machine."""
        name = self._get_selected_vm()
        if not name: return
        def thread():
            dom = self.conn.lookupByName(name)
            dom.reboot() if dom.isActive() else dom.create()
            self.refresh_vm_list()
        threading.Thread(target=thread).start()
    def open_console(self):
        """Opens VNC console for selected VM."""
        name = self._get_selected_vm()
        if not name: return
        dom = self.conn.lookupByName(name)
        if not dom.isActive(): return self.log_to_console("VM is not running", True)
        xml = ET.fromstring(dom.XMLDesc(0))
        graphics = xml.find('.//graphics[@type="vnc"]')
        port = graphics.get('port')
        subprocess.Popen(["vncviewer", f"localhost:{port}"])
    def _get_selected_vm(self):
        """Returns the name of the selected VM."""
        item = self.vm_tree.selection()
        return self.vm_tree.item(item)['values'][0] if item else None
    def delete_vm(self):
        """Deletes the selected virtual machine and optionally its disk."""
```

```
        name = self._get_selected_vm()
        if not name or not messagebox.askyesno("Delete", f"Delete VM '{name}'?"): return
        def thread():
            dom = self.conn.lookupByName(name)
            xml = ET.fromstring(dom.XMLDesc(0))
            disk = xml.find('.//disk/source')
            if dom.isActive(): dom.destroy()
            dom.undefine()
            if disk is not None and messagebox.askyesno("Disk", "Delete disk file?"):
                os.remove(disk.get('file'))
            self.refresh_vm_list()
        threading.Thread(target=thread).start()
    def create_vm(self):
        """Creates a new VM with selected configuration."""
        # Implementation omitted for brevity; would follow existing pattern with subprocess + libvirt XML
creation
        pass


    def on_closing(self):
        """Handles application close and cleanup."""
        if self.conn:
            try: self.conn.close()
            except: pass
        self.root.destroy()


def main():
    root = tk.Tk()
    app = EnhancedHypervisorManagerGUI(root)
    root.protocol("WM_DELETE_WINDOW", app.on_closing)
    root.mainloop()
if __name__ == "__main__":
    main()
```

APPENDIX 2

1. Project Title Map with Program Outcomes and Program Specific Outcomes

Project Title	**Engineering Program Outcomes (POs)**	**Department Specific Outcomes (DSOs)**
Simplified Hypervisor	PO1, PO2, PO3, PO4, PO5, PO6, PO9, PO10, PO11, PO12	PSO1, PSO2

Explanation-

The "Simplified Hypervisor" project aligns with POs by fostering engineering knowledge (PO1), problem analysis (PO2), design/development (PO3), investigation (PO4), tool usage (PO5), societal impact (PO6), teamwork (PO9), communication (PO10), project management (PO11), and lifelong learning (PO12), while addressing DSOs through advanced system design (PSO1) and virtualization expertise (PSO2) in computer engineering.

2. Project Categorization

Project Title	**Project Categorization**
Simplified Hypervisor	Product based

Explanation-

Product-based: The project focuses on developing a tangible product that can be used by end users

3. Project Map with SDG Goals

The project aligns with the following SDG goals:

- SDG 9: Industry, Innovation, and Infrastructure – The hypervisor project contributes to technological innovation by enhancing virtualization capabilities, which are essential for cloud computing, IT infrastructure, and efficient resource utilization.

- SDG 12: Responsible Consumption and Production – By enabling efficient resource management through virtualization, the project reduces hardware dependency, leading to lower energy consumption and electronic waste.

Signature of Supervisor :

Designation

Dated :

APPENDIX 3

Certificate to be issued by the supervisor to issue Similarity Index Check Report

I have checked the soft copy of project-3 report submitted by _____ Roll Nos. _____ on the Topic _____ _____ . The report in PDF format consisting of _____ number of pages and _____ number of words has been checked by me on Turnitin on _____ (Date), which consists of Similarity Index of overall _____ percentage.

Signature of Supervisor :

Designation

Dated :

TURNITIN SIMILARITY REPORT (ONLY FRONT PAGE)